

Lecture 32.Tables

Joining Tables

Joining Two Tables

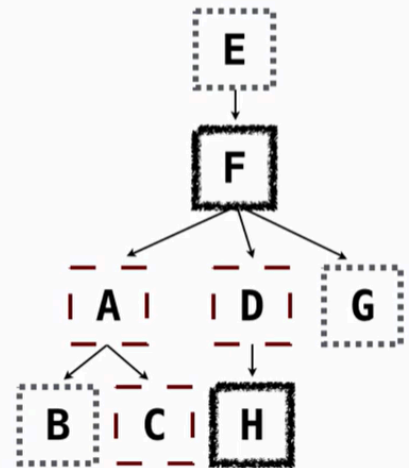
Two tables **A** & **B** are joined by a comma to yield all combos of a row from **A** & a row from **B**

```
create table dogs as
select "abraham" as name, "long" as fur union
select "barack"          , "short"      union
select "clinton"         , "long"       union
select "delano"           , "long"       union
select "eisenhower"      , "short"     union
select "fillmore"        , "curly"      union
select "grover"           , "short"     union
select "herbert"          , "curly";

create table parents as
select "abraham" as parent, "barack" as child union
select "abraham"          , "clinton"      union
...;
```

Select the parents of curly-furred dogs

```
select parent from parents, dogs
where child = name and fur = "curly";
```



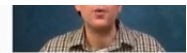
```
sqlite> select * from parents,dogs;
abraham|barack|abraham|long
abraham|barack|barack|short
abraham|barack|clinton|long
abraham|barack|delano|long
abraham|barack|eisenhower|short
abraham|barack|fillmore|curly
abraham|barack|grover|short
abraham|barack|herbert|curly
abraham|clinton|abraham|long
abraham|clinton|barack|short
abraham|clinton|clinton|long
abraham|clinton|delano|long
abraham|clinton|eisenhower|short
abraham|clinton|fillmore|curly
abraham|clinton|grover|short
abraham|clinton|herbert|curly
delano|herbert|abraham|long
delano|herbert|barack|short
delano|herbert|clinton|long
delano|herbert|delano|long
delano|herbert|eisenhower|short
delano|herbert|fillmore|curly
delano|herbert|grover|short
delano|herbert|herbert|curly
eisenhower|fillmore|abraham|long
```

```
create table dogs as
select "abraham" as name, "long" as fur union
select "barack"          , "short"      union
select "clinton"         , "long"       union
select "delano"           , "long"       union
select "eisenhower"      , "short"     union
select "fillmore"        , "curly"      union
select "grover"           , "short"     union
select "herbert"          , "curly";

create table parents as
select "abraham" as parent, "barack" as child union
select "abraham"          , "clinton"      union
select "delano"           , "herbert"      union
select "fillmore"         , "abraham"      union
select "fillmore"         , "delano"       union
select "fillmore"         , "grover"      union
select "eisenhower"       , "fillmore";
```

Aliases and Dot Expressions

Joining a Table with Itself



Two tables may share a column name; dot expressions and aliases disambiguate column values

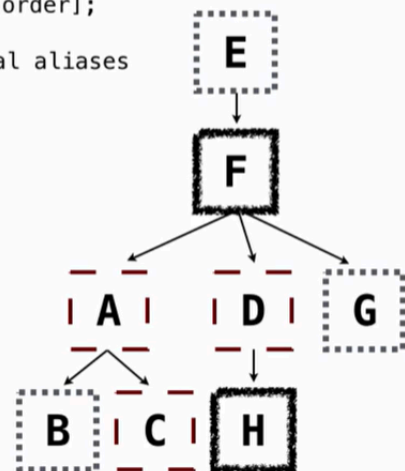
```
select [columns] from [table] where [condition] order by [order];
```

[table] is a comma-separated list of table names with optional aliases

Select all pairs of siblings

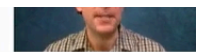
```
select a.child as first, b.child as second
from parents as a, parents as b
where a.parent = b.parent and a.child < b.child;
```

First	Second
barack	clinton
abraham	delano
abraham	grover
delano	grover



as you can see here, we use aliases and dot expressions to specify different tables/instances' columns.

Joining Multiple Tables



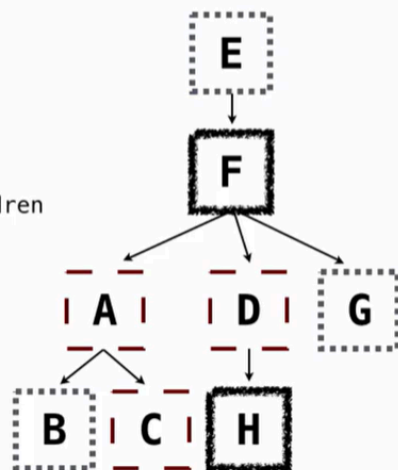
Multiple tables can be joined to yield all combinations of rows from each

```
create table grandparents as
select a.parent as granddog, b.child as granpup
from parents as a, parents as b
where b.parent = a.child;
```

Select all grandparents with the same fur as their grandchildren

Which tables need to be joined together?

```
select granddog from grandparents, dogs as c, dogs as d
where granddog = c.name and
      granpup = d.name and
      c.fur = d.fur;
```



```
create table grandparents as
  select a.parent as granddog, b.child as grandpup
     from parents as a, parents as b
     where a.child = b.parent;
```

```
select granddog from grandparents, dogs as c, dogs as d
  where c.fur = d.fur and
        c.name = granddog and
        d.name = grandpup;
```

Numerical Expressions

Numerical Expressions

Expressions can contain function calls and arithmetic operators

```
[expression] as [name], [expression] as [name], ...
```

```
select [columns] from [table] where [expression] order by [expression];
```

Combine values: +, -, *, /, %, and, or

Transform values: abs, round, not, -

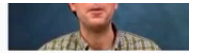
Compare values: <, <=, >, >=, <>, !=, =

```
sql> create table distances as
...>     select a.name as first, b.name as second,
...>         60*(b.latitude - a.latitude) as distance
...>         from cities as a, cities as b;
sql> select * from distances;
```

first	second	distance
Berkeley	Berkeley	0
Berkeley	New York	0
Berkeley	Cambridge	240
Berkeley	Minneapolis	420
New York	Berkeley	0
New York	New York	0
New York	Cambridge	240
New York	Minneapolis	420
Cambridge	Berkeley	-240
Cambridge	New York	-240
Cambridge	Cambridge	0
Cambridge	Minneapolis	180
Minneapolis	Berkeley	-420
Minneapolis	New York	-420
Minneapolis	Cambridge	-180
Minneapolis	Minneapolis	0

String Expressions

String Expressions



String values can be combined to form longer strings



```
sqlite> select "hello," || " world";  
hello, world
```

Basic string manipulation is built into SQL, but differs from Python



```
sqlite> create table phrase as select "hello, world" as s;  
sqlite> select substr(s, 4, 2) || substr(s, instr(s, " ")+1, 1) from phrase;  
low
```

Strings can be used to represent structured values, but doing so is rarely a good idea



```
sqlite> create table lists as select "one" as car, "two,three,four" as cdr;  
sqlite> select substr(cdr, 1, instr(cdr, ",")-1) as cadr from lists;  
two
```